

Tools en Agentes e IA: MCP vs Integración Directa

Guía didáctica para el aula · CEIABD 2025-2026

1. ¿Qué es una Tool en el contexto de los LLMs?

Una tool (herramienta) es una función externa que un modelo de lenguaje (LLM) puede invocar durante una conversación para obtener información o realizar acciones que no puede hacer por sí mismo. Ejemplos habituales:

- Consultar el tiempo meteorológico actual
- Realizar operaciones matemáticas con precisión
- Buscar información en una base de datos
- Enviar un correo electrónico o crear una tarea

El LLM no ejecuta la función directamente: detecta cuándo es necesaria, genera la llamada con los parámetros adecuados, y un sistema externo (el agente o cliente) es quien la ejecuta y devuelve el resultado.

2. Dos formas de integrar Tools

2.1 Tools internas al Agente

En este enfoque, las funciones están definidas directamente en el mismo código del agente. El LLM recibe la descripción de las herramientas en cada llamada y decide cuándo usarlas.

Esquema conceptual:

```
[Usuario] → [Agente + Tools definidas aquí] → [LLM]
                ↑
            obtener_tiempo(), calcular()...
```

Ejemplo de código:

```
tools = [
    {"type": "function", "function": {
        "name": "obtener_tiempo",
        "description": "Obtiene el clima de una ciudad",
        "parameters": {...}
    }}
]
```

```
]
```

```
def obtener_tiempo(ciudad):  
    return {"temperatura": "31°C", "condicion": "nublado"}
```

Ventajas:

- Sencillo de implementar y depurar
- Menos infraestructura necesaria
- Ideal para proyectos pequeños o prototipos

Inconvenientes:

- El código de las tools está acoplado al agente
- Si se tienen varios agentes, hay que duplicar el código
- Difícil de mantener a escala

2.2 Tools via Servidor MCP

Model Context Protocol (MCP) es un estándar abierto que permite exponer herramientas como un servicio independiente. El agente se conecta al servidor MCP y descubre las tools disponibles de forma dinámica.

Esquema conceptual:

```
[Usuario] → [Agente] → [LLM]  
           ↓  
       [Servidor MCP]  
       obtener_tiempo()  
       calcular()  
       buscar_en_bd() ...
```

Ejemplo de servidor MCP con FastMCP:

```
from fastmcp import FastMCP  
mcp = FastMCP("mi-servidor")
```

```
@mcp.tool  
def obtener_tiempo(ciudad: str) -> dict:  
    """Obtiene el clima actual para una ciudad."""  
    ...
```

```
if __name__ == "__main__":  
    mcp.run(transport="stdio")
```

Ventajas:

- Las tools son reutilizables por cualquier agente o LLM compatible

- Un único servidor para múltiples clientes
- Mantenimiento centralizado: cambias el servidor y todos los agentes se actualizan
- Favorece la modularidad y el trabajo en equipo

Inconvenientes:

- Mayor complejidad inicial (servidor independiente)
- Requiere gestionar la comunicación entre procesos

3. Tabla Comparativa

Aspecto	Tools Internas	Tools via MCP
Dónde vive el código	Dentro del agente	Servidor independiente
Reutilizable	No, solo ese agente	Sí, cualquier cliente MCP
Mantenimiento	Cambias N agentes	Cambias 1 servidor
Complejidad	Baja	Media-Alta
Escalabilidad	Limitada	Alta
Cuándo usarlo	Proyectos pequeños, un agente	Varios agentes, equipos, producción

4. La Analogía para Recordarlo

Tools internas: como llevar las herramientas en el bolsillo. Funcionan, pero solo tú puedes usarlas.

 **Servidor MCP:** como tener un taller compartido. Cualquiera puede ir a buscar la herramienta que necesite.

5. El Flujo Completo con MCP y un LLM

Este es el flujo cuando se integra un servidor MCP con un LLM local (por ejemplo, vía LM Studio):

```
1. El usuario hace una pregunta en lenguaje natural
```

```
→ "¿Qué tiempo hace en Valladolid?"
```

```
2. El agente envía la pregunta al LLM junto con la lista de tools disponibles
```

```
3. El LLM decide que necesita usar obtener_tiempo()  
→ tool_call: obtener_tiempo({"ciudad": "Valladolid"})
```

```
4. El agente llama al servidor MCP con esos parámetros  
→ resultado: {"temperatura": "31°C", "condicion": "nublado", ...}
```

```
5. El agente devuelve el resultado al LLM
```

```
6. El LLM redacta la respuesta final en lenguaje natural  
→ "En Valladolid hay 31°C con cielo parcialmente nublado..."
```

El servidor MCP no cambia en ningún momento. Esa es la gran ventaja: creas las herramientas una vez y cualquier agente compatible puede utilizarlas.

6. Conclusión

Ambos enfoques son válidos y complementarios. La elección depende del contexto:

- Para aprender, prototipar o proyectos de un solo agente → tools internas
- Para producción, equipos o múltiples agentes → servidor MCP

Frase clave: *"El servidor MCP sabe hacer cosas. El agente sabe cuándo y por qué hacerlas."*